

Introduction to Machine Learning

K -means and Clustering Problems

Solutions

Prof. Sundeep Rangan

Problem 1

You are given five data samples:

i	1	2	3	4	5
x_{i1}	0	1	0	2	2
x_{i2}	0	0	1	2	3

(a) Draw the five points.

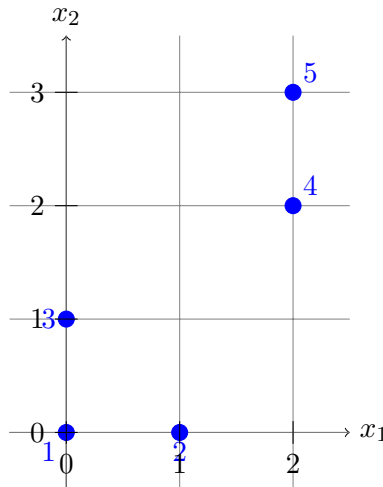


Figure 1: Plot of the five data points

(b) Starting with $K = 2$ cluster centers at $(0,0)$ and $(1,0)$, what are the cluster assignments and new cluster centers after one iteration of K -means?

Step 1: Assignment Step

For each point, compute the squared Euclidean distance to both cluster centers and assign to the nearest center.

- **Point 1** (0,0):
 - Distance to center 1 (0,0): $d^2 = (0 - 0)^2 + (0 - 0)^2 = 0$
 - Distance to center 2 (1,0): $d^2 = (0 - 1)^2 + (0 - 0)^2 = 1$
 - Assignment: **Cluster 1**
- **Point 2** (1,0):
 - Distance to center 1 (0,0): $d^2 = (1 - 0)^2 + (0 - 0)^2 = 1$
 - Distance to center 2 (1,0): $d^2 = (1 - 1)^2 + (0 - 0)^2 = 0$
 - Assignment: **Cluster 2**
- **Point 3** (0,1):
 - Distance to center 1 (0,0): $d^2 = (0 - 0)^2 + (1 - 0)^2 = 1$
 - Distance to center 2 (1,0): $d^2 = (0 - 1)^2 + (1 - 0)^2 = 2$
 - Assignment: **Cluster 1**
- **Point 4** (2,2):
 - Distance to center 1 (0,0): $d^2 = (2 - 0)^2 + (2 - 0)^2 = 8$
 - Distance to center 2 (1,0): $d^2 = (2 - 1)^2 + (2 - 0)^2 = 5$
 - Assignment: **Cluster 2**
- **Point 5** (2,3):
 - Distance to center 1 (0,0): $d^2 = (2 - 0)^2 + (3 - 0)^2 = 13$
 - Distance to center 2 (1,0): $d^2 = (2 - 1)^2 + (3 - 0)^2 = 10$
 - Assignment: **Cluster 2**

Cluster assignments:

- **Cluster 1:** Points 1 and 3: $\{(0,0), (0,1)\}$
- **Cluster 2:** Points 2, 4, and 5: $\{(1,0), (2,2), (2,3)\}$

Step 2: Update Step

Compute new cluster centers as the mean of assigned points:

- **New Center 1:**

$$\mu_1 = \frac{(0,0) + (0,1)}{2} = \left(\frac{0+0}{2}, \frac{0+1}{2} \right) = (0, 0.5)$$

- **New Center 2:**

$$\mu_2 = \frac{(1,0) + (2,2) + (2,3)}{3} = \left(\frac{1+2+2}{3}, \frac{0+2+3}{3} \right) = \left(\frac{5}{3}, \frac{5}{3} \right) \approx (1.67, 1.67)$$

Answer: After one iteration:

- **Cluster assignments:** Cluster 1: $\{1, 3\}$, Cluster 2: $\{2, 4, 5\}$
- **New cluster centers:** $\mu_1 = (0, 0.5)$, $\mu_2 = (5/3, 5/3) \approx (1.67, 1.67)$

Problem 2

K-means for outlier detection. Write a function for outlier detection:

Solution

```
def outlier_detect(Xtr, Xts, nc, t):
    """
    Detect outliers using K-means clustering.

    Parameters:
    Xtr: Training data (N_tr x D matrix)
    Xts: Test data (N_ts x D matrix)
    nc: Number of clusters
    t: Distance threshold

    Returns:
    outlier: Binary array (N_ts x 1), 1 if outlier, 0 otherwise
    """
    from sklearn.cluster import KMeans
    import numpy as np

    # Perform K-means on training data
    km = KMeans(n_clusters=nc)
    km.fit(Xtr)

    # Get cluster centers (nc x D matrix)
    centers = km.cluster_centers_ # Shape: (nc, D)

    # Compute distances from each test point to all cluster centers
    # Xts: (N_ts, D), centers: (nc, D)
    # We want: distances[i, j] = ||Xts[i] - centers[j]||^2

    # Using broadcasting:
    # Xts[:, None, :] has shape (N_ts, 1, D)
    # centers[None, :, :] has shape (1, nc, D)
    # Difference has shape (N_ts, nc, D)
    diff = Xts[:, None, :] - centers[None, :, :]
    distances_sq = np.sum(diff**2, axis=2) # Shape: (N_ts, nc)

    # Find minimum distance from each test point to any cluster center
    min_distances = np.sqrt(np.min(distances_sq, axis=1)) # Shape: (N_ts,)

    # Mark as outlier if distance > threshold
    outlier = (min_distances > t).astype(int)
```

```
return outlier
```

Explanation:

- We use broadcasting to compute all pairwise distances efficiently without loops
- `Xts[:, None, :]` reshapes to $(N_{ts}, 1, D)$ for broadcasting
- `centers[None, :, :]` reshapes to $(1, nc, D)$ for broadcasting
- The difference gives us (N_{ts}, nc, D) , then we sum over the last axis to get squared distances
- We find the minimum distance for each test point and compare to threshold

Problem 3

Initialization. Write a few lines of python code to initialize K -means by selecting K random samples of the training data as the cluster centers. Make sure you do not pick the same sample twice.

Solution

```
import numpy as np

# Method 1: Using np.random.choice (recommended)
n = Xtr.shape[0] # Number of training samples
indices = np.random.choice(n, size=K, replace=False)
mu_init = Xtr[indices, :]

# Method 2: Using random permutation
indices = np.random.permutation(n)[:K]
mu_init = Xtr[indices, :]

# Method 3: Manual approach (less efficient but clear)
indices = []
while len(indices) < K:
    idx = np.random.randint(0, n)
    if idx not in indices:
        indices.append(idx)
mu_init = Xtr[indices, :]
```

Explanation:

- `np.random.choice(n, size=K, replace=False)` selects K unique indices from $[0, n)$
- `replace=False` ensures no duplicates
- `Xtr[indices, :]` extracts the corresponding rows from the training data
- Method 1 is the most efficient and Pythonic approach

Problem 4

Clustering as pre-processing. Suppose we want to cluster data and then fit a linear model in each cluster. You are given training data X_{tr}, y_{tr} and test data X_{ts}, y_{ts} for a regression problem. Write code to do the following:

- Perform K -means clustering on the training data X_{tr} with a given number nc clusters;
- In each cluster in the training data, fit a linear model.
- Compute the predicted outputs $\hat{y}_{ts}$ and mean squared error of the model on the test data.

Solution

```
from sklearn.cluster import KMeans
from sklearn.linear_model import LinearRegression
import numpy as np

# Perform K-means clustering on training data
km = KMeans(n_clusters=nc)
km.fit(Xtr)

# Get cluster assignments for training data
cluster_train = km.predict(Xtr) # Shape: (N_tr,)

# Get cluster assignments for test data
cluster_test = km.predict(Xts) # Shape: (N_ts,)

# Initialize list to store regression models for each cluster
regressors = [LinearRegression() for _ in range(nc)]

# Fit a linear model in each cluster
for k in range(nc):
    # Find training samples in cluster k
    mask_train = (cluster_train == k)
    Xtr_k = Xtr[mask_train, :]
    ytr_k = ytr[mask_train]

    # Fit linear model for cluster k
    if Xtr_k.shape[0] > 0: # Check if cluster has samples
        regressors[k].fit(Xtr_k, ytr_k)

# Predict on test data
yhat_ts = np.zeros(yts.shape)
for k in range(nc):
    # Find test samples in cluster k
    mask_test = (cluster_test == k)
```

```

if np.any(mask_test) and Xtr[cluster_train == k].shape[0] > 0:
    # Predict using the model for cluster k
    yhat_ts[mask_test] = regressors[k].predict(Xts[mask_test, :])

# Compute mean squared error
mse = np.mean((yts - yhat_ts)**2)

```

Explanation:

- We create a list of LinearRegression objects, one for each cluster
- For each cluster, we extract the training samples belonging to that cluster and fit a linear model
- For prediction, we assign each test sample to its cluster and use the corresponding regression model
- We handle edge cases where a cluster might have no training samples

Problem 5

Figure 2 shows a set of samples to be clustered. Show the results from K-means algorithm in successive iterations, starting with the initial centroids indicated in the figure. You can do nearest neighbor partition and centroid update approximately by “eyeballing”.

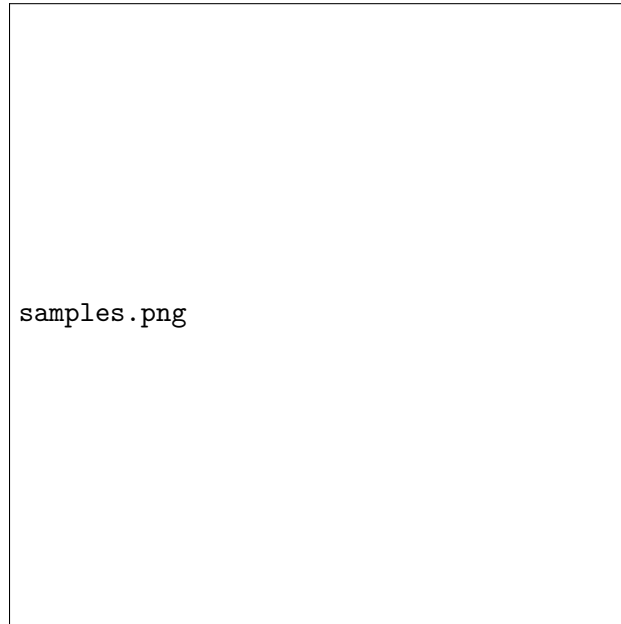


Figure 2: The set of samples to be clustered

Solution

Note: Without access to the actual figure, I’ll provide a general approach for solving this problem:

1. **Initialization:** Identify the initial centroids marked in Figure 2
2. **Iteration 1 - Assignment Step:**
 - For each data point, compute the distance to all centroids
 - Assign each point to the nearest centroid
 - Draw the Voronoi diagram (partition boundaries) showing which points belong to which cluster
3. **Iteration 1 - Update Step:**
 - Compute the new centroid for each cluster as the mean of all points assigned to it
 - Mark the new centroid positions
4. **Iteration 2:** Repeat the assignment and update steps with the new centroids
5. **Continue** until convergence (centroids no longer change significantly)

General Procedure:

- Draw the current partition boundaries (perpendicular bisectors between centroids)
- Color-code points by their cluster assignment
- Compute and mark new centroids
- Repeat until the algorithm converges

Problem 6

Suppose you have conducted a clustering analysis for a dataset with each sample described by D features, and you used K-means algorithm to derive K clusters and determined the cluster model parameters (including the centroids of the K clusters). Given a test dataset containing N samples, you want to classify each sample into one of the cluster using the nearest neighbor rule. How many computations are needed? For simplicity, for this and all following problems, only count multiplications (consider the square operation as multiplication).

Solution

For each test sample, we need to:

1. Compute the squared distance to each of the K cluster centers
2. Find the minimum distance

For one test sample:

- Distance to one cluster center: D subtractions + D multiplications (squaring) + $(D - 1)$ additions
- Since we only count multiplications: D multiplications per cluster center
- For K cluster centers: $K \times D$ multiplications
- Finding the minimum: negligible (comparisons, not multiplications)

For N test samples:

$$\text{Total multiplications} = N \times K \times D$$

Answer: NKD multiplications

Problem 7

Suppose you are given N samples each described by D features, and you are asked to cluster them into K clusters using the K-means algorithm. Suppose you run the K-means iteration T times. How many computations are needed?

Solution

Each iteration of K-means consists of two steps:

Step 1: Assignment Step

- For each of N samples, compute distance to K cluster centers
- Each distance computation: D multiplications (for squared differences)
- Total: $N \times K \times D$ multiplications per iteration

Step 2: Update Step

- For each of K clusters, compute the mean of assigned points
- For cluster k with n_k points:
 - Sum n_k vectors of dimension D : $n_k \times D$ additions (not counted)
 - Divide by n_k : D divisions (not counted as multiplications)
 - No multiplications needed for this step

Total for T iterations:

$$\text{Total multiplications} = T \times N \times K \times D$$

Answer: $TNKD$ multiplications

Problem 8 (Optional)

Suppose you have conducted a clustering analysis for a dataset with each sample described by D features, and you used EM-GMM algorithm to derive K clusters and determined the cluster model parameters (including the prior probabilities, centroids and covariance matrices of the K clusters). Given a test dataset containing N samples, you want to classify each sample into the cluster that has the highest posterior probability. How many computations are needed?

Solution

For each test sample $\mathbf{x}$, we compute the posterior probability for each cluster k :

$$P(k | \mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \mathbf{S}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_j, \mathbf{S}_j)}$$

where $\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \mathbf{S}_k)$ is the multivariate Gaussian PDF.

For one cluster k :

Computing $\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \mathbf{S}_k)$ requires:

- Compute $\mathbf{x} - \boldsymbol{\mu}_k$: D subtractions (not counted)
- Compute $(\mathbf{x} - \boldsymbol{\mu}_k)^T \mathbf{S}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)$:
 - Matrix-vector product $\mathbf{S}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)$: $D \times D$ multiplications
 - Dot product: D multiplications
 - Total: $D^2 + D$ multiplications
- Determinant computation: Typically $O(D^3)$ but for pre-computed $\mathbf{S}_k^{-1}$, we can pre-compute $\det(\mathbf{S}_k)$
- Exponential and normalization: Not multiplications

For K clusters:

- Computing all K Gaussian PDFs: $K \times (D^2 + D)$ multiplications
- Computing the denominator (sum): Negligible (additions)
- Finding maximum: Negligible (comparisons)

For N test samples:

$$\text{Total multiplications} = N \times K \times (D^2 + D) = NK(D^2 + D)$$

Answer: $NK(D^2 + D)$ multiplications

Problem 9 (Optional)

Suppose you are given N samples each described by D features, and you are asked to cluster them into K clusters using the EM-GMM algorithm. Suppose you run the EM iteration T times. How many computations are needed?

Solution

Each EM iteration consists of two steps:

E-step (Expectation):

- For each sample i and cluster k , compute the responsibility:

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \mathbf{S}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \mathbf{S}_j)}$$

- For one sample and one cluster: $D^2 + D$ multiplications (same as Problem 8)
- For N samples and K clusters: $N \times K \times (D^2 + D)$ multiplications

M-step (Maximization):

- Update prior probabilities: $\pi_k = \frac{1}{N} \sum_{i=1}^N \gamma_{ik}$ (no multiplications, just additions and one division per k)
- Update centroids: $\boldsymbol{\mu}_k = \frac{\sum_{i=1}^N \gamma_{ik} \mathbf{x}_i}{\sum_{i=1}^N \gamma_{ik}}$
 - For each cluster k : $N \times D$ multiplications (for $\gamma_{ik} \mathbf{x}_i$)
 - Total for K clusters: $K \times N \times D$ multiplications
- Update covariance matrices: $\mathbf{S}_k = \frac{\sum_{i=1}^N \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T}{\sum_{i=1}^N \gamma_{ik}}$
 - For each sample i in cluster k : Compute outer product $(\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T$
 - Outer product: D^2 multiplications
 - Weighted by γ_{ik} : D^2 more multiplications
 - For N samples and K clusters: $N \times K \times 2D^2$ multiplications

Total per iteration:

$$NK(D^2 + D) + KND + 2NKD^2 = NK(3D^2 + 2D)$$

Total for T iterations:

$$\text{Total multiplications} = T \times NK(3D^2 + 2D) = TNK(3D^2 + 2D)$$

Answer: $TNK(3D^2 + 2D)$ multiplications